

Merge Sort

Origem e história

O Merge Sort foi criado por John von Neumann em 1945. A ideia de dividir um problema em subproblemas menores, resolvê-los recursivamente e depois combinar (fazer o *merge*) se tornou um dos exemplos mais clássicos do paradigma de divisão e conquista (*divide and conquer*), influenciando algoritmos posteriores.

Abordagem

O Merge Sort é um algoritmo de divisão e conquista. Isso significa que ele resolve o problema seguindo três etapas:

- **Dividir:** o vetor é dividido ao meio, gerando duas metades.
- **Conquistar:** cada metade é ordenada recursivamente, aplicando o mesmo processo até restar apenas um elemento, que já está "ordenado" por definição.
- **Combinar (*merge*):** as duas metades já ordenadas são intercaladas em um único vetor ordenado.

Como funciona

Dado um vetor de n elementos:

1. Se o vetor tem 0 ou 1 elemento, já está ordenado. Esse é o caso base da recursão.
2. Caso contrário, divide-se o vetor em duas metades: esquerda e direita.
3. Aplica-se o Merge Sort recursivamente em cada metade.
4. As duas metades ordenadas são mescladas: compara-se o primeiro elemento de cada metade, coloca-se o menor no vetor de saída e repete-se o processo até que todos os elementos estejam em ordem.

Explicação simples: a analogia das cartas

Imagine que você tem um monte de cartas de baralho bagunçadas na mão e quer organizá-las.

A sequência funciona assim:

1. Você divide o monte ao meio, entrega metade para um amigo e fica com a outra metade.
2. Cada pessoa divide sua parte ao meio novamente. Isso se repete até cada pessoa ficar com apenas uma carta.
3. Uma carta sozinha já está "organizada", pois não há outra carta para comparar.
4. Começa o caminho de volta: cada dupla compara suas cartas e forma um pequeno monte de duas cartas em ordem.
5. Os montes de duas cartas são juntados, comparando carta por carta, para formar montes maiores e ordenados.
6. O processo se repete até sobrar um único monte, totalmente ordenado.

A parte de dividir o monte corresponde à divisão. Nessa etapa, ninguém organiza nada: apenas reparte as cartas. A parte de juntar comparando as cartas corresponde ao *merge*, que é quando a ordenação realmente acontece.

Exemplo: [6, 3, 8, 2]

1. Divide [6, 3, 8, 2] em [6, 3] e [8, 2].
2. Divide [6, 3] em 6 e 3, e [8, 2] em 8 e 2.
3. Cada parte tem apenas um elemento.
4. Compara 6 e 3, formando [3, 6].
5. Compara 8 e 2, formando [2, 8].
6. Por fim, compara [3, 6] e [2, 8], formando [2, 3, 6, 8].

Cada "encontro de dois montes" é um *merge* separado. No exemplo, o *merge* acontece três vezes: duas vezes juntando pares de cartas e uma vez juntando os dois montes de duas cartas no vetor final.

Complexidade

Cenário	Complexidade
Melhor caso	$O(n \log n)$
Caso médio	$O(n \log n)$
Pior caso	$O(n \log n)$

O que essa complexidade significa

A notação $O(n \log n)$ descreve como o tempo de execução do algoritmo cresce em relação ao tamanho da entrada (n), no pior caso.

- O fator $\log n$ vem da etapa de divisão: como o vetor é sempre dividido ao meio, são necessárias $\log_2(n)$ divisões até chegar aos casos base, que são vetores de tamanho 1. Isso forma uma árvore de recursão com $\log n$ níveis.
- O fator n vem da etapa de *merge*: em cada nível da recursão, o trabalho total de mesclar as sublistas é proporcional a n , pois cada elemento é percorrido uma vez.
- Multiplicando os dois fatores, ou seja, n elementos processados em cada um dos $\log n$ níveis, obtém-se o total de $n \log n$ operações.

O espaço $O(n)$ refere-se à memória auxiliar usada para armazenar os vetores temporários usados durante o *merge*.

Estabilidade

O Merge Sort é um algoritmo estável. Isso significa que, se dois elementos possuem o mesmo valor de comparação, a ordem inicial entre eles é preservada no resultado final.

Essa característica é garantida porque, durante o *merge*, quando os elementos de ambos os lados são iguais, a implementação padrão prioriza o elemento que veio da metade esquerda. Por consequência da divisão, esse elemento aparecia antes no vetor original.

In-place ou não?

O Merge Sort, em sua implementação clássica, não é *in-place*. Ele requer memória auxiliar proporcional ao tamanho do vetor ($O(n)$) para armazenar os elementos durante a etapa de *merge*, pois não é possível mesclar

duas sublistas ordenadas diretamente no mesmo espaço de memória sem sobrescrever dados que ainda serão lidos.

```
// Função auxiliar para intercalar (merge) duas metades ordenadas
void merge(int arr[], int inicio, int meio, int fim) {
    int n1 = meio - inicio + 1;
    int n2 = fim - meio;

    // Cria vetores temporários para as duas metades
    int* esquerda = new int[n1];
    int* direita = new int[n2];

    // Copia os dados para os vetores temporários
    for (int i = 0; i < n1; i++) {
        esquerda[i] = arr[inicio + i];
    }
    for (int j = 0; j < n2; j++) {
        direita[j] = arr[meio + 1 + j];
    }

    // Intercala os vetores temporários de volta em arr[inicio..fim]
    int i = 0; // Índice inicial da primeira metade
    int j = 0; // Índice inicial da segunda metade
    int k = inicio; // Índice inicial do vetor combinado

    while (i < n1 && j < n2) {
        if (esquerda[i] <= direita[j]) {
            arr[k] = esquerda[i];
            i++;
        } else {
            arr[k] = direita[j];
            j++;
        }
        k++;
    }

    // Copia os elementos restantes de esquerda[], se houver
    while (i < n1) {
        arr[k] = esquerda[i];
        i++;
        k++;
    }

    // Copia os elementos restantes de direita[], se houver
    while (j < n2) {
        arr[k] = direita[j];
        j++;
        k++;
    }

    // Libera a memória alocada dinamicamente
    delete[] esquerda;
```

```
        delete[] direita;
    }

    // Função principal do Merge Sort
    void mergeSort(int arr[], int inicio, int fim) {
        // Caso base: sub-arranjo de tamanho 0 ou 1 já está ordenado
        if (inicio >= fim) {
            return;
        }

        // Evita overflow de (inicio + fim) / 2
        int meio = inicio + (fim - inicio) / 2;

        // Divide e conquista: ordena as duas metades
        mergeSort(arr, inicio, meio);
        mergeSort(arr, meio + 1, fim);

        // Combina as duas metades ordenadas
        merge(arr, inicio, meio, fim);
    }
```